

MSI Circuits:

Those circuits who have 12-99 logic gates internally.

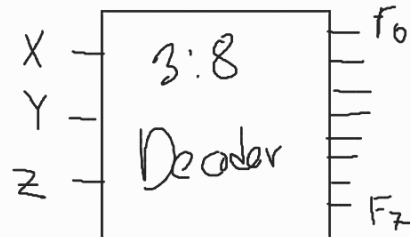
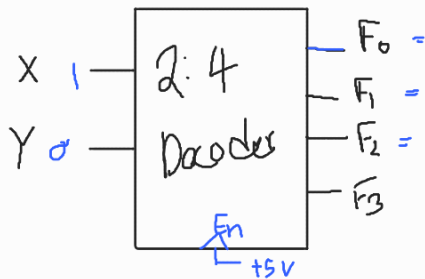
1. Decoder ✓
2. DeMultiplexer (DEMUX)
3. Multiplexer (MUX) ✓
4. Encoder ✓
5. Priority Encoder ✓

Implementation

1. Decoder:

n input

2^n output



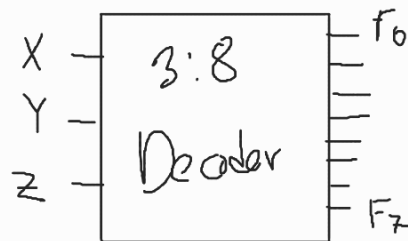
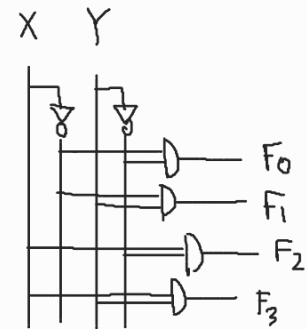
X	Y	F_0	F_1	F_2	F_3
0	0	1	0	0	0
0	1	0	1	0	0
1	0	0	0	1	0
1	1	0	0	0	1

$$F_0 = X'Y'$$

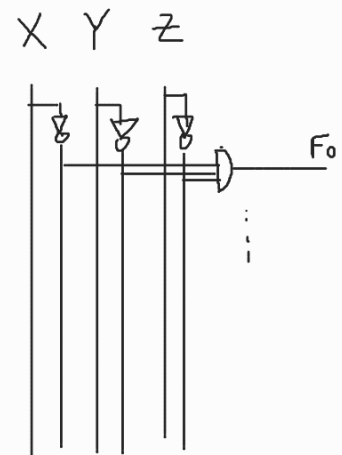
$$F_1 = X'Y$$

$$F_2 = XY'$$

$$F_3 = XY$$

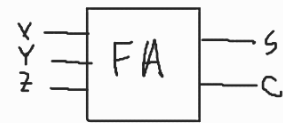


X	Y	Z	F0	F1	F2	F3	F4	F5	F6	F7	
0	0	0	1								$F_0 = X'Y'Z'$
0	0	1		1							$F_1 = X'Y'Z$
0	1	0			1						$F_2 = X'YZ'$
0	1	1				1					$F_3 = X'YZ$
1	0	0					1				$F_4 = XY'Z'$
1	0	1						1			$F_5 = XY'Z$
1	1	0							1		$F_6 = XYZ'$
1	1	1								1	$F_7 = XYZ$



We can implement any function with decoder.

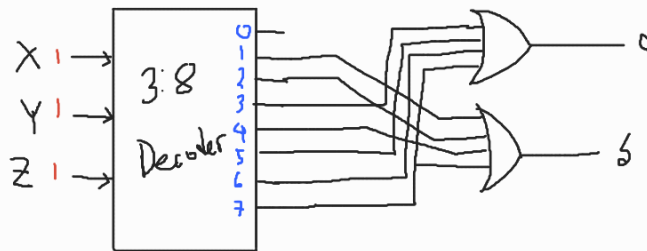
* Design a full adder using a 3:8 Decoder



	x	y	z	C	S
0	0	0	0	0	0
1	0	0	1	0	1
2	0	1	0	0	1
3	0	1	1	1	0
4	1	0	0	0	1
5	1	0	1	1	0
6	1	1	0	1	0
7	1	1	1	1	1

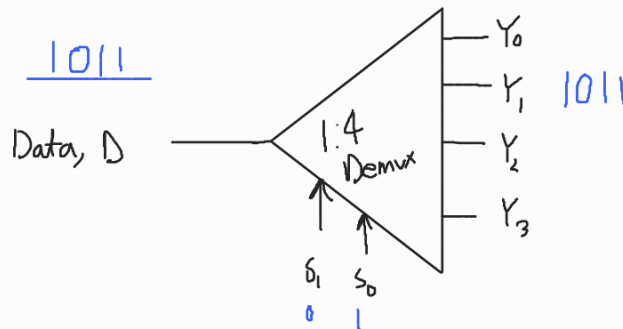
$$C = \Sigma(3, 5, 6, 7)$$

$$S = \Sigma(1, 2, 4, 7)$$



Demultiplexer: (DEMUX)

Given an input line (Data, D) and a set of selection lines, DEMUX will direct data from input to a selected output line.



n selection,
 2^n output
 1 Data line

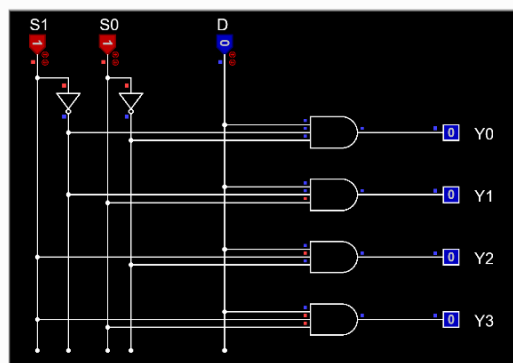
S_1	S_0	Y_0	Y_1	Y_2	Y_3
0	0	D	0	0	0
0	1	0	D	0	0
1	0	0	0	D	0
1	1	0	0	0	D

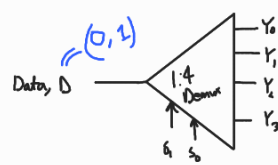
$$Y_0 = D \cdot S_1' \cdot S_0'$$

$$Y_1 = D \cdot S_1' \cdot S_0$$

$$Y_2 = D \cdot S_1 \cdot S_0'$$

$$Y_3 = D \cdot S_1 \cdot S_0$$





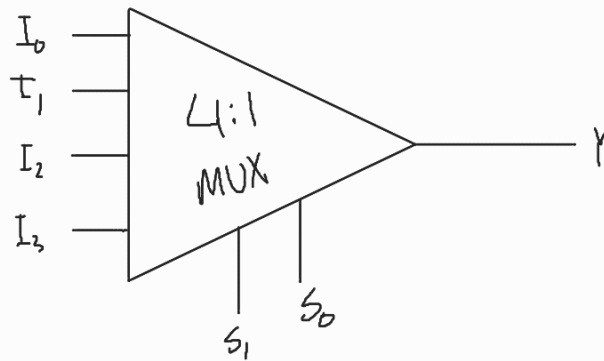
Data input = 1
 Selector = n
 Output = 2^n

Multiplexer (MUX):

Such circuit where 2^n data input, n selection line & 1 output line.

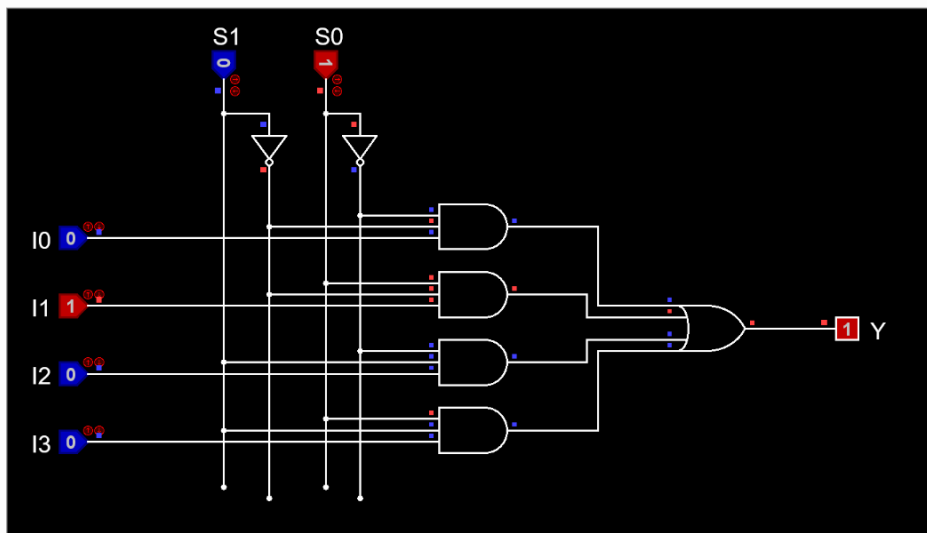
Aka, Data Selector

Data input = 2^n
 Selector = n
 Output = 1



s_1	s_0	Y
0	0	$I_0 = 1$
0	1	$I_1 = 1$
1	0	$I_2 = 1$
1	1	$I_3 = 1$

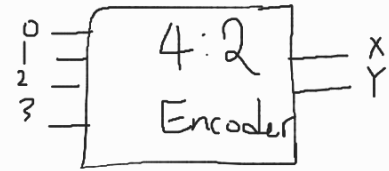
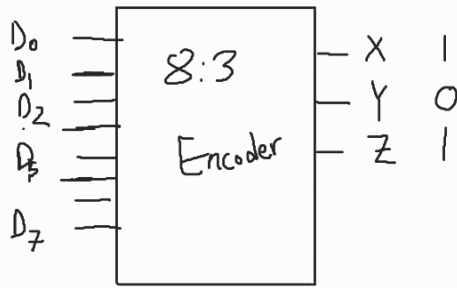
$$Y = I_0 \cdot s_1' s_0' + I_1 \cdot s_1' s_0 + I_2 \cdot s_1 s_0' + I_3 \cdot s_1 s_0$$



Encoder:
Reverse of Decoder

Output = n

Input = 2^n

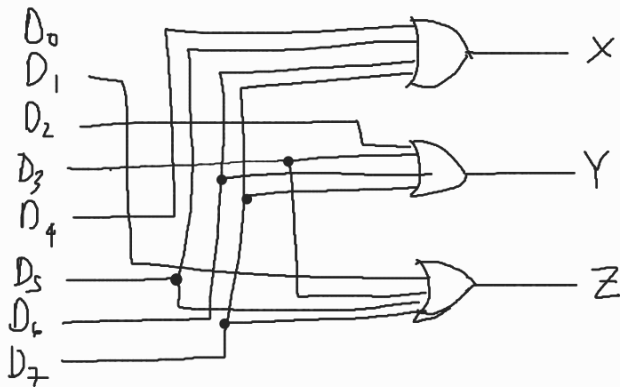


D_0	D_1	D_2	D_3	D_4	D_5	D_6	D_7	X	Y	Z
1	0	0	0	0	0	0	0	0	0	0
0	1	0	0	0	0	0	0	0	0	1
		1	0	0	0	0	0	0	1	0
			1	0	0	0	0	0	1	1
				1	0	0	0	1	0	0
					1	0	0	1	0	1
						1	0	1	1	0
							1	1	1	1

$$X = D_4 + D_5 + D_6 + D_7$$

$$Y = D_2 + D_3 + D_6 + D_7$$

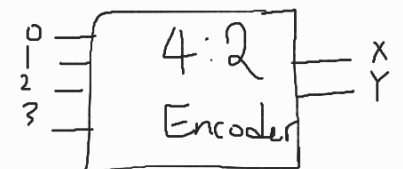
$$Z = D_1 + D_3 + D_5 + D_7$$



8:3 Encoder

4:2 Encoder

D_0	D_1	D_2	D_3	X	Y
1				0	0
	1			0	1
		1		1	0
			1	1	1



$$X = D_2 + D_3$$

$$Y = D_1 + D_3$$

Encoder

Normal (Only one 1 in input)

Priority Encoder

LSB ✓
MSB ✓

D_3	D_2	D_1	D_0	X	Y
0	1	1	1	0	0
0	1	1	0	1	0

Design a 4:2 priority encoder(MSB)

Input = D_3, D_2, D_1, D_0

Output = X, Y

	D_3	D_2	D_1	D_0	X	Y
0	0	0	0	0	x	x
1	0	0	0	1	0	0
2	0	0	1	0	0	1
3	0	0	1	1	0	1
4	0	1	0	0	1	0
5	0	1	0	1	1	0
6	0	1	1	0	1	0
7	0	1	1	1	1	0
8	1	0	0	0	1	1
9	1	0	0	1	1	1
10	1	0	1	0	1	1
11	1	0	1	1	1	1
12	1	1	0	0	1	1
13	1	1	0	1	1	1
14	1	1	1	0	1	1
15	1	1	1	1	1	1

$$X = \sum(4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15) + d(0)$$

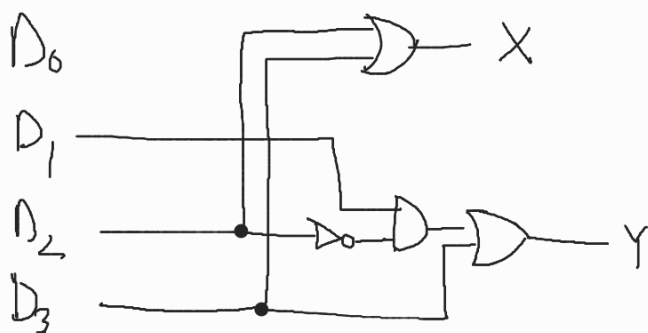
$$Y = \sum(2, 3, 8, 9, 10, 11, 12, 13, 14, 15) + d(0)$$

$D_3 D_2$	$D_1 D_0'$	$D_1' D_0$	$D_1 D_0$	$D_1 D_0'$
$D_3' D_2'$	x	0	1	3
$D_3' D_2$	4	5	7	6
$D_3 D_2$	12	13	15	14
$D_3 D_2'$	8	9	11	10

$$X = D_2 + D_3$$

$D_3 D_2$	$D_1 D_0'$	$D_1' D_0$	$D_1 D_0$	$D_1 D_0'$
$D_3' D_2'$	x	0	1	3
$D_3' D_2$	4	5	7	6
$D_3 D_2$	12	13	15	14
$D_3 D_2'$	8	9	11	10

$$Y = D_3 + D_2' D_1$$



4:2 Priority Encoder (MSB)

Design a 4:2 priority encoder(LSB)

Input = D3,D2,D1,D0

Output = X,Y

	D3	D2	D1	D0	X	Y
0	0	0	0	0	x	x
1	0	0	0	1	0	0
2	0	0	1	0	0	1
3	0	0	1	1	0	0
4	0	1	0	0	1	0
5	0	1	0	1	0	0
6	0	1	1	0	0	1
7	0	1	1	1	0	0
8	1	0	0	0	1	1
9	1	0	0	1	0	0
10	1	0	1	0	0	1
11	1	0	1	1	0	0
12	1	1	0	0	1	0
13	1	1	0	1	0	0
14	1	1	1	0	0	1
15	1	1	1	1	0	0

$$X = \sum(4, 8, 12) + d(0)$$

$$Y = \sum(2, 6, 10, 14) + d(0)$$

Encoder Related Problems:

Priority Encoder Design:

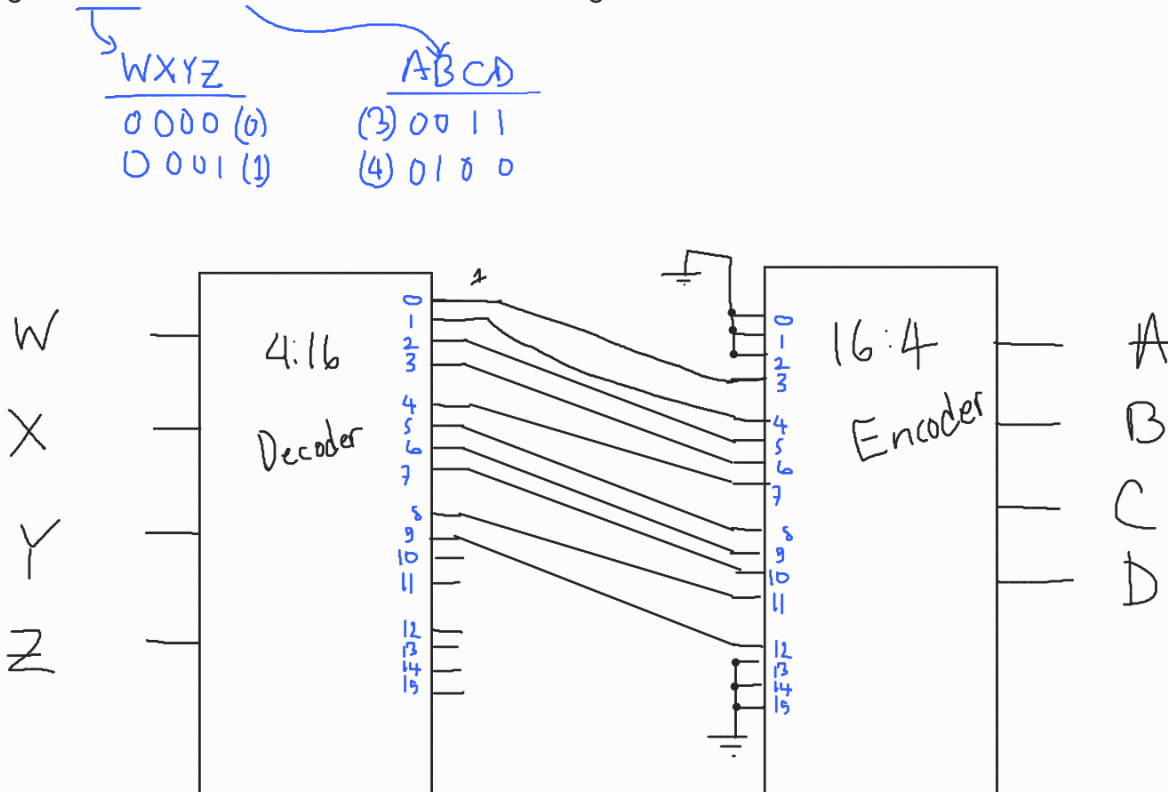
1. Design a 4:2 priority encoder(MSB)

2. Design a 4:2 priority encoder(LSB)

3. Design a 4:2 priority encoder that will give priority to MSB if the number of 1 in input is even, otherwise it will give priority to LSB.

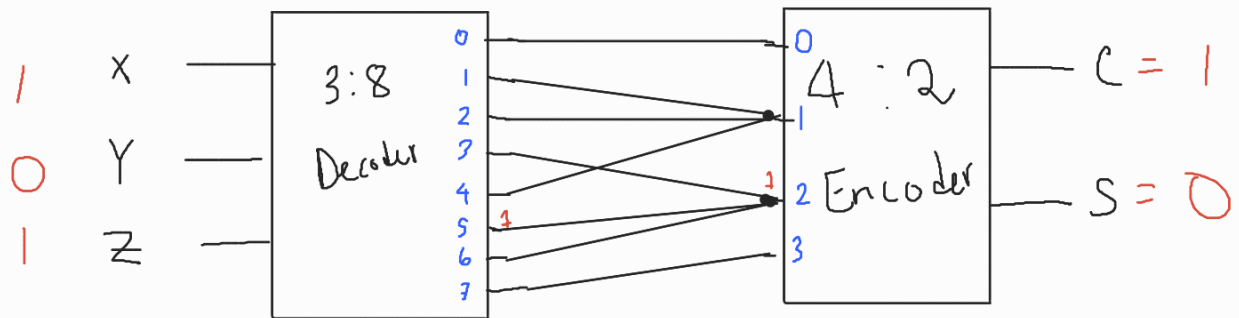
Decoder Encoder Related Problems:

1. Design a BCD to Excess 3 Code Converter using 4x16 Decoder and 16x4 Encoder



Design a Full Adder using Decoder and Encoder.

	3				2		
	x	y	z		C	S	
0	0	0	0		0	0	0
1	0	0	1		0	1	1
2	0	1	0		0	1	1
3	0	1	1		1	0	2
4	1	0	0		0	1	1
5	1	0	1		1	0	2
6	1	1	0		1	0	2
7	1	1	1		1	1	3



Design a Half adder using Decoder and Encoder.